Mealvana Nutrition Algorithm v4

Introduction

This document specifies the **implemented** nutrition algorithm as of v4, reflecting the Rachel/Xuan-corrected formulas that replaced the original v3 design. It documents exactly what the `generate-macros-v3` edge function computes and what the Dart client sends/parses.

Key Changes from v3 Spec to v4

Area	v3 Spec (Original)	v4 (Implemented, Rachel-Corrected)
Pre-workout carbs	Cumulative 3-window with intensity-weighted g/kg	Linear: 1 g/kg per hour before, single calculation
Pre-workout intensity	Direct multiplier in carb formulas	Intensity does NOT affect pre-workout carbs
During gut training	Hard absorption caps (40/60/100 g/hr)	Multipliers (0.7×/1.0×/1.2×) on entire band
During-workout intensity	Positions within band using intensity	Midpoint of scaled band (intensity not used)
Transitions	Weight-based (0.3 g/kg / 0.35 g/kg)	Fixed values (T1: 20g, T2: 25g)
Brick duration bands	Cumulative duration for band lookup	Per-segment duration for band lookup
Brick bike/run bonuses	+20% bike front-load, -25% run penalty	Not implemented (future consideration)

Research basis: - ISSN Position Stand: Nutrient Timing (Kerksick et al. 2017) - Jeukendrup A. (2014). “A Step Towards Personalized Sports Nutrition” - Rachel/Xuan corrections documented in `/docs/new_macros/new_notes_rachel.md`

Example Personas

Persona	Description	Weight	Gut Training	Sweat Rate	Sweat Sodium
Elena	Elite female runner, 28yo, competitive marathoner	52 kg	high	medium	medium
Marcus	Male age-grouper, 42yo, experienced triathlete	75 kg	moderate	medium	medium
Sarah	Female beginner, 35yo, training for first half-marathon	82 kg	low	light	low

Example Workouts

Workout A: Moderate 2-Hour Run

- **Duration:** 120 min | **Sport:** run
- **Intensity:** 30% Z1-2, 50% Z3, 20% Z4-5
- **Temperature:** 22°C | **Humidity:** 55%
- **Hours before:** 3.0 | **Fasted:** No

Workout B: Easy 45-Minute Recovery Run

- **Duration:** 45 min | **Sport:** run
- **Intensity:** 90% Z1-2, 10% Z3, 0% Z4-5
- **Temperature:** 18°C | **Humidity:** 50%
- **Hours before:** 1.5 | **Fasted:** No

Workout C: Long 4-Hour Bike Ride

- **Duration:** 240 min | **Sport:** bike
- **Intensity:** 40% Z1-2, 45% Z3, 15% Z4-5
- **Temperature:** 28°C | **Humidity:** 70%
- **Hours before:** 3.5 | **Fasted:** No

Input Variables

Variable	Unit	Description	Required
`weight`	number	Athlete's body weight	Yes
`weight_unit`	`kg` or `lb`	Weight unit	Yes
`hours_before`	number	Hours between last meal and workout start	Yes
`is_fasted`	bool	Whether athlete is training fasted	Yes
`activity_type`	enum	`running`, `cycling`, `swimming`, or `brick`	Yes
`intensity_distribution`	object	`{zone_low, zone_mid, zone_high}` (0-1 fractions, sum=1)	Optional (estimated from MET if absent). Not used for during-carb targeting.
`gut_training`	enum	`low`, `moderate`, or `high`	Yes
`sweat_rate_category`	enum	`light`, `medium`, or `heavy`	Yes
`sweat_sodium`	enum	`low`, `medium`, or `high`	Yes
`temp_c`	°C	Environmental temperature	Optional (default: 20)
`humidity_pct`	%	Relative humidity	Optional (default: 60)

Sport-specific inputs:

Sport	Additional Fields
Running	`run_distance`, `run_distance_unit`, `run_pace`, `run_pace_unit`
Cycling	`distance_miles`, `speed_mph`, `terrain`
Swimming	`distance_meters`, `pace_per_100m_seconds`, `pool_or_open_water`, `water_temp_c`
Brick	`brick_segments[]` (each with sport, order, duration_minutes, intensity, sport-specific fields), `segment_order[]`

v3→v4 input change: v3 had separate `has_full_meal`, `has_snack`, `has_topup` booleans. v4 replaces these with a single `hours_before` value. The meal type is derived: - `hours_before >= 2.5` → full_meal - `hours_before >= 1.0` → snack - `hours_before < 1.0` → top_up - `is_fasted = true` → fasted (all zeros)

Lookup Tables

TABLE 1: Duration → Carb Band (g/hr) — DURING EXERCISE

Absolute g/hr ranges. NOT body-weight dependent.

Duration	band_low	band_high	Source
< 60 min	0	30	Mouth rinse sufficient
60-90 min	30	60	Maintain blood glucose
90 min - 2.5 hr	45	60	Delay glycogen depletion
2.5 - 4 hr	60	90	Multiple transportable carbs
> 4 hr	80	100	Maximum sustainable

Source: Jeukendrup 2014, ISSN Position Stand (Kerksick 2017)

Implementation: `getDurationCarbBand()` in edge function, lines 197-214.

TABLE 2: Sport → Carb Ceiling (g/hr) — DURING EXERCISE

Applied AFTER gut training scaling.

Sport	carb_ceiling	Source
Running	70	GI jostling limits tolerance
Cycling	120	Stable platform, highest tolerance
Swimming	0	Cannot eat during activity

Implementation: `getSportCarbCeiling()` in edge function, lines 232-237.

TABLE 3: Gut Training → Multiplier (NOT caps)

CRITICAL CHANGE from v3 spec: These are MULTIPLIERS applied to the entire duration band, not hard absorption caps.

gut_training	multiplier	Example: 90min band (45-60) becomes
low	0.7×	31.5-42 g/hr
moderate	1.0×	45-60 g/hr (unchanged)
high	1.2×	54-72 g/hr

Why multipliers, not caps (Rachel correction #4): - v3 spec used hard caps (low=40, moderate=60, high=100 g/hr) that flattened the band to a single ceiling - Multipliers preserve the range structure so the midpoint still scales up or down with gut training - Research supports that gut training affects absorption proportionally, not as a binary cutoff

Implementation: `getGutTrainingMultiplier()` in edge function, lines 222-227.

TABLE 4: Sweat Rate → Base Rate (L/hr)

sweat_rate	base_rate	Source
light	0.75 L/hr	Baker et al. 2017: lower quartile
medium	1.25 L/hr	Baker et al. 2017: median
heavy	2.0 L/hr	Baker et al. 2017: upper quartile

Implementation: `baseSweatRateFromCategory()` in edge function, lines 70-75.

TABLE 5: Sweat Sodium → Concentration (mg/L)

sweat_sodium	concentration	Source
low	550 mg/L	Baker et al. 2017: ~25th percentile
medium	925 mg/L	Baker et al. 2017: ~50th percentile
high	1150 mg/L	Baker et al. 2017: ~75th percentile

Implementation: `sodiumConcentrationFromCategory()` in edge function, lines 80-85.

TABLE 6: Environment Classification

Condition	temp_c	humidity_pct	sweat_multiplier	label
Cool	≤10	any	0.85	`cool`
Temperate	≤20	≤60	1.0	`temperate`
Warm	≤25	or >60 & ≤75	1.1	`warm`
Hot	≤30	or >75 & ≤85	1.2	`hot`
Very Hot	>30	or >85	1.3	`very_hot`

Note: The sweat rate environmental adjustment uses a separate formula:

```
temp_adjustment = 1.0 + max(0, (temp_c − 20) × 0.04)
actual_sweat_rate = base_rate × temp_adjustment
```

Implementation: `classifyEnvironment()` and `calculateActualSweatRate()` in edge function.

Derived Values

```
duration_hours = duration / 60
```

```
actual_sweat_rate = base_sweat_rate × (1.0 + max(0, (temp_c − 20) × 0.04))
```

Actual sweat rates by persona and workout:

Persona	Sweat Base	Workout A (22°C)	Workout B (18°C)	Workout C (28°C)
Elena (medium)	1.25 L/hr	1.35 L/hr	1.25 L/hr	1.65 L/hr
Marcus (medium)	1.25 L/hr	1.35 L/hr	1.25 L/hr	1.65 L/hr
Sarah (light)	0.75 L/hr	0.81 L/hr	0.75 L/hr	0.99 L/hr

Note: Workout B (18°C) — temp < 20, so temp_adjustment = 1.0 (no reduction below baseline).

Phase 1: Pre-Workout

How Pre-Workout Works in v4

CRITICAL CHANGE from v3: Pre-workout is now a **single calculation** based on `hours_before`, not three cumulative windows.

Rachel Correction #1: Total pre-workout carbs = 1 g/kg per hour of pre-workout window (linear). NOT stacked per window.

Rachel Correction #2: Intensity does NOT affect carbs per kg. It only influences the suggested window length (UX concern, not algorithm).

Carbs (g)

Formula:

```
if is_fasted:
    pre_carbs = 0
else:
    carb_per_kg = max(0.5, min(hours_before, 4.0))
    pre_carbs = round(weight_kg × carb_per_kg)
```

Explanation: Linear relationship — 1 g/kg per hour, with a floor of 0.5 g/kg (even <30 min gets something) and a ceiling of 4.0 g/kg (matches ISSN's 1-4 g/kg range).

Examples:

Persona	Workout A (3.0h before)	Workout B (1.5h before)	Workout C (3.5h before)
Elena (52 kg)	**156g** (3.0 g/kg)	**78g** (1.5 g/kg)	**182g** (3.5 g/kg)
Marcus (75 kg)	**225g** (3.0 g/kg)	**113g** (1.5 g/kg)	**263g** (3.5 g/kg)
	246g (3.0 g/kg)	**123g** (1.5 g/kg)	**287g** (3.5 g/kg)

Persona	Workout A (3.0h before)	Workout B (1.5h before)	Workout C (3.5h before)
Sarah (82 kg)			

No intensity adjustment. The same hours_before produces the same g/kg regardless of workout difficulty.

Meal Type Classification

The `hours_before` value determines the meal type, which affects protein, fat, sodium, and hydration:

hours_before	meal_type	Protein	Fat	Sodium	Hydration
≥ 2.5	`full_meal`	0.25 g/kg	0.4 g/kg	baseSodium + envBump	6.5 ml/kg
≥ 1.0	`snack`	0.15 g/kg	5g (fixed)	(baseSodium + envBump) × 0.5	5.5 ml/kg
< 1.0	`top_up`	0g	0g	envBump + 100	250ml (fixed)
fasted	`fasted`	0g	0g	0mg	0ml

Where:

```
baseSodium:
    low sweat_sodium:    300 mg
    medium sweat_sodium:  450 mg
    high sweat_sodium:    600 mg

envBump:
    hot or very_hot:     100 mg
    else:                  0 mg
```

Implementation: `calculatePreWorkoutMacros()` in edge function, lines 120-185.

Pre-Workout Protein (g)

Formula (varies by meal type):

```
if full_meal:   protein = round(weight_kg × 0.25)
if snack:       protein = round(weight_kg × 0.15)
if top_up:      protein = 0
if fasted:      protein = 0
```

Examples (Full Meal, hours_before ≥ 2.5):

Persona	All Workouts
Elena (52 kg)	**13g**
Marcus (75 kg)	**19g**
Sarah (82 kg)	**21g**

Pre-Workout Fat (g)

Formula (varies by meal type):

```
if full_meal:   fat = round(weight_kg × 0.4)
if snack:       fat = 5
if top_up:      fat = 0
if fasted:      fat = 0
```

Examples (Full Meal):

Persona	All Workouts
Elena (52 kg)	**21g**
Marcus (75 kg)	**30g**
Sarah (82 kg)	**33g**

Pre-Workout Sodium (mg)

Formula (varies by meal type):

```
if full_meal:   sodium = baseSodium + envBump
if snack:       sodium = round((baseSodium + envBump) × 0.5)
if top_up:      sodium = envBump + 100
if fasted:      sodium = 0
```

Examples (Full Meal):

Persona	Workout A (22°C, temperate)	Workout B (18°C, temperate)	Workout C (28°C, hot)
Elena (medium Na)	**450mg**	**450mg**	**550mg**
Marcus (medium Na)	**450mg**	**450mg**	**550mg**
Sarah (low Na)	**300mg**	**300mg**	**400mg**

Pre-Workout Hydration (mL)

Formula (varies by meal type):

```
if full_meal:   hydration = round(weight_kg × 6.5)
if snack:       hydration = round(weight_kg × 5.5)
if top_up:      hydration = 250
if fasted:      hydration = 0
```

Examples (Full Meal):

Persona	All Workouts
Elena (52 kg)	**338mL**
Marcus (75 kg)	**488mL**
Sarah (82 kg)	**533mL**

Pre-Workout Summary

Workout A (Full Meal, 3h before, 22°C):

Persona	Carbs	Protein	Fat	Sodium	Hydration
Elena (52 kg)	156g	13g	21g	450mg	338mL
Marcus (75 kg)	225g	19g	30g	450mg	488mL
Sarah (82 kg)	246g	21g	33g	300mg	533mL

Fasted Training Guidelines

Eligibility (implemented in `FastedToggle.checkEligibility()`):

```
fasted_eligible:
    if isSwimming:                          false
    if conversationalPct < 70:              false    #
~pct_zone_high > 0.3
    if estimatedDurationMinutes > 75:       false
    else:                                   true
```

If `is_fasted = true`: - Pre-workout: all macros = 0 - Post-workout: carbs × 1.2 multiplier, protein 0.35 g/kg (instead of 0.3)

Phase 2: During Exercise

Carbs (g/hr)

Formula (v4 — Rachel-corrected):

```
if sport = swim:
    during_carb_rate = 0

else:
    # Step 1: Get duration band (TABLE 1)
    [band_low, band_high] = getDurationCarbBand(duration_min)

    # Step 2: Apply gut training MULTIPLIER to entire band (TABLE
3)
    gut_mult = getGutTrainingMultiplier(gut_training)
    scaled_low = band_low × gut_mult
    scaled_high = band_high × gut_mult

    # Step 3: Use midpoint of scaled band
    carb_rate = (scaled_low + scaled_high) / 2

    # Step 4: Apply sport ceiling (TABLE 2)
    final_rate = min(carb_rate, sport_ceiling)

during_carb_total = final_rate × duration_hours
```

Key difference from v3 spec: v3 used `min(carb_in_band, carb_ceiling, absorption_cap)` where absorption_cap was a hard number. v4 applies gut training as a multiplier to the band, uses the midpoint of the scaled band, then applies the sport ceiling.

Worked Example — Workout A (2hr run, 120 min):

Step	Elena (high gut)	Marcus (mod gut)	Sarah (low gut)
1. Band (90-150min)	45-60	45-60	45-60
2. Gut multiplier	×1.2 → 54-72	×1.0 → 45-60	×0.7 → 31.5-42
3. Midpoint	(54 + 72) / 2 = **63.0**	(45 + 60) / 2 = **52.5**	(31.5 + 42) / 2 = **36.8**
4. Sport ceiling (70)	min(63.0, 70) = **63.0 g/hr**	min(52.5, 70) = **52.5 g/hr**	min(36.8, 70) = **36.8 g/hr**
Total (2hr)	**126g**	**105g**	**74g**

All examples (g/hr rate, total in parentheses):

Persona	Workout A (2hr run)	Workout B (45min run)	Workout C (4hr bike)
Elena (high gut)	**63.0 g/hr** (126g)	**18.0 g/hr** (14g)	**108.0 g/hr** (432g)
Marcus (mod gut)	**52.5 g/hr** (105g)	**15.0 g/hr** (11g)	**90.0 g/hr** (360g)
	36.8 g/hr (74g)	**10.5 g/hr** (8g)	**63.0 g/hr** (252g)

Persona	Workout A (2hr run)	Workout B (45min run)	Workout C (4hr bike)
Sarah (low gut)			

Workout B: <60min band (0-30). Midpoint applied after gut multiplier (18/15/10.5 g/hr). Mouth rinse still acceptable. Workout C: >4hr band (80-100), bike ceiling 120 → gut training is the limiting factor.

Implementation: `calculateDuringWorkoutCarbRate()` in edge function, lines 248-285.

Sodium (mg/hr)

Formula:

```
sodium_rate = round(actual_sweat_rate × sodium_concentration × 0.6)
sodium_total = round(sodium_rate × duration_hours)
```

Target: 60% of sweat sodium losses during exercise.

Examples:

Persona	Workout A (22°C)	Workout B (18°C)	Workout C (28°C)
Elena (medium)	**749 mg/hr** (1498mg)	**694 mg/hr** (520mg)	**916 mg/hr** (3663mg)
Marcus (medium)	**749 mg/hr** (1498mg)	**694 mg/hr** (520mg)	**916 mg/hr** (3663mg)
Sarah (low)	**267 mg/hr** (534mg)	**248 mg/hr** (186mg)	**327 mg/hr** (1307mg)

Implementation: `calculateDuringWorkoutHydration()` in edge function, lines 290-324.

Hydration (mL/hr)

Formula:

```
hydration_rate = round(actual_sweat_rate × 1000 × 0.75)
hydration_total = round(hydration_rate × duration_hours)
```

Target: 75% of sweat rate replacement during exercise.

Examples:

Persona	Workout A (22°C)	Workout B (18°C)	Workout C (28°C)
		938 mL/hr (703mL)	

Persona	Workout A (22°C)	Workout B (18°C)	Workout C (28°C)
Elena (medium)	**1013 mL/hr** (2025mL)		**1238 mL/hr** (4950mL)
Marcus (medium)	**1013 mL/hr** (2025mL)	**938 mL/hr** (703mL)	**1238 mL/hr** (4950mL)
Sarah (light)	**608 mL/hr** (1215mL)	**563 mL/hr** (422mL)	**743 mL/hr** (2970mL)

During-Exercise Protein and Fat

```
during_protein_rate = 0 g/hr    # v4 does not implement ultra-endurance protein
during_fat_rate = 0 g/hr        # v4 does not implement ultra-endurance fat
```

Note: v3 spec mentioned 3 g/hr protein and 2 g/hr fat for >4hr workouts. This is NOT implemented in the edge function.

Phase 3: Post-Workout

Carbs (g)

Formula:

```
duration_multiplier = 1.2 if duration_hours > 2, else 1.0
fasted_multiplier = 1.2 if is_fasted, else 1.0

post_carbs = round(weight_kg × duration_multiplier × fasted_multiplier)
```

Examples:

Persona	Workout A (2hr)	Workout B (45min)	Workout C (4hr)
Elena (52 kg)	**52g** (1.0 g/kg)	**52g** (1.0 g/kg)	**62g** (1.2 g/kg)
Marcus (75 kg)	**75g** (1.0 g/kg)	**75g** (1.0 g/kg)	**90g** (1.2 g/kg)
Sarah (82 kg)	**82g** (1.0 g/kg)	**82g** (1.0 g/kg)	**98g** (1.2 g/kg)

If fasted: multiply by 1.2. Elena fasted Workout B = 62g.

Implementation: `calculatePostWorkoutCarbs()` in edge function, lines 334-346.

Protein (g)

Formula:

```
protein_per_kg = 0.35 if is_fasted, else 0.30
post_protein = round(weight_kg × protein_per_kg)
```

Examples:

Persona	Normal	After Fasted Training
Elena (52 kg)	**16g**	**18g**
Marcus (75 kg)	**23g**	**26g**
Sarah (82 kg)	**25g**	**29g**

Implementation: `calculatePostWorkoutProtein()` in edge function, lines 352-358.

Fat (g)

Formula:

```
post_fat = round(weight_kg × 0.2)
```

Persona	All Workouts
Elena (52 kg)	**10g**
Marcus (75 kg)	**15g**
Sarah (82 kg)	**16g**

Implementation: `calculatePostWorkoutFat()` in edge function, lines 363-365.

Sodium (mg) — Post-Workout

Formula:

```
total_sodium_lost = actual_sweat_rate × sodium_concentration × duration_hours
during_sodium = round(actual_sweat_rate × sodium_concentration × 0.6 × duration_hours)
sodium_deficit = total_sodium_lost − during_sodium

post_sodium = max(300, min(700, round(sodium_deficit × 0.5)))
```

Replace 50% of sodium deficit, clamped to 300-700 mg range.

Implementation: `calculatePostWorkoutHydration()` in edge function, lines 371-399.

Hydration (mL) — Post-Workout

Formula:

```
total_fluid_lost = actual_sweat_rate × 1000 × duration_hours
hydration_deficit = total_fluid_lost − during_hydration_total

post_hydration = round(max(500, hydration_deficit × 1.5))
```

Replace 150% of hydration deficit, minimum 500mL.

Implementation: `calculatePostWorkoutHydration()` in edge function, lines 371-399.

Phase 4: Transitions (Brick Workouts Only)

Fixed Transition Values

CHANGE from v3 spec: v3 used weight-based transitions (T1: 0.3 g/kg, T2: 0.35 g/kg). v4 uses **fixed values** for simplicity.

Transition	Carbs	Protein	Fat	Sodium	Water	Timing
T1 (after first segment)	20g	0g	0g	150mg	200mL	Within 5-10 min after first segment
T2 (after second segment)	25g	0g	0g	100mg	150mL	Final 5-10 min of second segment

These values do NOT vary by body weight, temperature, or any other variable.

Implementation: `calculateBrickMacrosV3()` in edge function, lines 962-982.

Brick-Specific Design Decisions

Per-segment duration bands (not cumulative): Each segment uses its OWN duration for the carb band lookup. A 30-min swim segment uses the <60min band, even if the total brick is 2.5 hours.

Rationale for per-segment (v4 decision): Simpler implementation, and each sport segment has its own GI characteristics regardless of cumulative time.

Swimming segments: Always 0 carbs, 0 sodium, 0 water during swimming. Athletes cannot eat/drink while swimming.

Post-workout for brick: Uses aggregate (total) duration across all segments for the carb per kg calculation. The post-workout hydration deficit calculation sums all during-segment hydration + transition hydration.

Future Considerations (Not Implemented)

The following were in the v3 spec but are NOT implemented. They need Rachel/Xuan input before adding:

- **Post-bike run penalty:** 25% reduction in run carb ceiling (70 → 52.5 g/hr) when running after a bike segment. Blog mentions "20-30% reduction in run carb targets after bike leg."
- **Bike front-load bonus:** +20% on bike carb rate when the bike is followed by a run segment. Rationale: last opportunity for high-volume fueling before the more GI-sensitive run.

Energy Calculations

Running MET (ACSM)

```
speed_mph = 60 / pace_min_per_mile
speed_m_per_min = speed_mph × 26.8224

if speed_mph >= 4.0:
    VO2 = 0.2 × speed_m_per_min + 3.5    # Running equation
else:
    VO2 = 0.1 × speed_m_per_min + 3.5    # Walking equation

MET = VO2 / 3.5
```

Cycling MET

Speed (kph)	MET	Terrain adjustment
≤16	6.0	rolling: ×1.1
≤19	8.0	hilly: ×1.25
≤22	10.0	
≤25	12.0	
≤30	14.0	
>30	16.0	

Swimming MET

Pace (sec/100m)	MET	Adjustments
≥180	6.0	Open water: ×1.15
≥150	8.0	Cold water (<20°C): ×1.1

Pace (sec/100m)	MET	Adjustments
≥120	10.0	Warm water (>28°C): ×0.95
≥90	11.0	
<90	13.0	

Calorie Calculations

```
# Gross (total energy including BMR)
calories_gross = round(MET × 3.5 × weight_kg / 200 × duration_min)

# Net (transport cost only)
Running:  net = round(1.0 × weight_kg × distance_km)
Cycling:  net = round(weight_kg × distance_km × cost_per_kg_km)
    cost_per_kg_km: ≤20kph=0.3, ≤25kph=0.35, ≤30kph=0.4,
>30kph=0.5
Swimming: net = round(3.5 × weight_kg × distance_km)
```

Implementation: Lines 401-497 in edge function.

Edge Function Response Format

Single-Sport Response


```
{
  "success": true,
  "macros": {
    "algorithm_version": "v3",
    "activity_type": "running",
    "duration_min": 120.0,
    "duration_h": 2.0,
    "distance_km": 19.312,
    "calories_gross_kcal": 1200,
    "calories_net_kcal": 980,
    "MET": 10.5,
    "intensity_distribution": { "zone_low": 0.3, "zone_mid": 0.5,
        "zone_high": 0.2 },

    "pre_run_carbs_g": 225,
    "pre_run_protein_g": 19,
    "pre_run_fat_g": 30,
    "pre_run_sodium_mg": 450,
    "pre_run_water_ml": 488,
    "pre_run_meal_type": "full_meal",

    "during_rate_g_per_h": 52.5,
    "during_total_g": 105,
    "during_band_low_g_per_h": 45,
    "during_band_high_g_per_h": 60,
```

```
    "during_gut_multiplier": 1.0,
    "during_sport_ceiling_g_per_h": 70,
    "during_sodium_rate_mg_per_h": 749,
    "during_sodium_total_mg": 1498,
    "during_water_rate_ml_per_h": 1013,
    "during_water_total_ml": 2025,

    "post_run_carbs_g": 75,
    "post_run_protein_g": 23,
    "post_run_fat_g": 15,
    "post_run_sodium_mg": 500,
    "post_run_water_ml": 1013,

    "sweat_rate_lph": 1.35,
    "sodium_conc_mg_per_l": 925,
    "environment_label": "warm",
    "environment_multiplier": 1.1
  }
}
```


Brick Response


```
{
  "success": true,
  "macros": {
    "algorithm_version": "v3",
    "activity_type": "brick",
    "duration_h": 2.0,
    "duration_min": 120.0,
    "distance_mi": 15.5,
    "distance_km": 24.9,
    "calories_gross_kcal": 1500,
    "calories_net_kcal": 1200,
    "phases": {
      "before": {
        "carbs_g": 188,
        "protein_g": 19,
        "fat_g": 30,
        "sodium_mg": 450,
        "water_ml": 488,
        "meal_type": "full_meal"
      },
      "during_segments": [
        { "segment_order": 0, "sport": "swimming",
        "duration_minutes": 30, "carbs_g": 0, "protein_g": 0,
        "fat_g": 0, "sodium_mg": 0, "water_ml": 0,
        "food_categories": ["during_swimming"] },
        { "segment_order": 1, "sport": "cycling",
        "duration_minutes": 60, "carbs_g": 45, "protein_g": 0,
        "fat_g": 0, "sodium_mg": 749, "water_ml": 1013,
        "food_categories": ["during_cycling"] },
```

```
        { "segment_order": 2, "sport": "running",
        "duration_minutes": 30, "carbs_g": 18, "protein_g": 0,
        "fat_g": 0, "sodium_mg": 375, "water_ml": 506,
        "food_categories": ["during_running"] }
      ],
      "transitions": [
        { "transition_name": "T1", "after_sport": "swimming",
        "before_sport": "cycling", "carbs_g": 20, "protein_g": 0,
        "fat_g": 0, "sodium_mg": 150, "water_ml": 200,
        "timing_note": "Within first 5-10 minutes after first
        segment", "food_categories": ["transition"] },
        { "transition_name": "T2", "after_sport": "cycling",
        "before_sport": "running", "carbs_g": 25, "protein_g": 0,
        "fat_g": 0, "sodium_mg": 100, "water_ml": 150,
        "timing_note": "Final 5-10 minutes of second segment",
        "food_categories": ["transition"] }
      ],
      "after": {
        "carbs_g": 75,
        "protein_g": 23,
        "sodium_mg": 500,
        "water_ml": 1013
      }
    }
  }
}
```

Dart Client Parsing

Single-Sport Parsing (MacroGenerationService)

The Dart client maps edge function fields to `MacroTargets`:

Edge Function Field	Dart Field	Domain Object
`pre_run_carbs_g`	`carbsG`	`PreRunMacros`
`pre_run_protein_g`	`proteinG`	`PreRunMacros`
`pre_run_fat_g`	`fatCapG`	`PreRunMacros`
`pre_run_water_ml`	`fluidsMl`	`PreRunMacros`
`pre_run_sodium_mg`	`sodiumMg`	`PreRunMacros`
`during_rate_g_per_h`	`carbRateGPerH`	`DuringRunMacros`
`during_total_g`	`carbTotalG`	`DuringRunMacros`
`during_water_rate_ml_per_h`	`fluidRateMlPerH`	`DuringRunMacros`
`during_water_total_ml`	`fluidTotalMl`	`DuringRunMacros`
`during_sodium_rate_mg_per_h`	`sodiumRateMgPerH`	`DuringRunMacros`
`during_sodium_total_mg`	`sodiumTotalMg`	`DuringRunMacros`
`post_run_carbs_g`	`carbsG`	`PostRunMacros`
`post_run_protein_g`	`proteinG`	`PostRunMacros`

Edge Function Field	Dart Field	Domain Object
post_run_fat_g	fatG	PostRunMacros

Bug fixed in v4: Prior to v4, the Dart parser read pre_run_protein_g_optional and pre_run_fat_g_cap (wrong field names), causing pre-workout protein and fat to silently be 0.

Brick Parsing (BrickMacroService)

Brick responses are normalized from the multi-phase structure to the standard 3-phase MacroTargets: - preRun ← phases.before - duringRun ← Sum of all phases.during_segments + all phases.transitions - postRun ← phases.after

LLM Fallback (LLMNutritionPlanService)

When generateLLMNutritionPlan() is called without pre-calculated MacroTargets, it computes its own estimates. **These are now aligned with v4 formulas:**

- Pre-workout: carb_per_kg = clamp(hours_before, 0.5, 4.0), meal-type-based protein/fat
- During: Duration band lookup with gut training multiplier, midpoint of scaled band, capped at sport ceiling
- Post-workout: Duration-dependent (1.0 or 1.2 g/kg), 0.3 g/kg protein

---

Appendix A: Complete Workout Summaries

Workout A: 2-Hour Moderate Run (22°C, 3h before)

Phase	Elena (52kg, high gut)	Marcus (75kg, mod gut)	Sarah (82kg, low gut)
Pre	156g carb, 13g pro, 21g fat, 450mg Na, 338mL	225g carb, 19g pro, 30g fat, 450mg Na, 488mL	246g carb, 21g pro, 33g fat, 300mg Na, 533mL
During	63.0g/hr (126g total)	52.5g/hr (105g total)	36.8g/hr (74g total)
During Na	749mg/hr (1498mg)	749mg/hr (1498mg)	267mg/hr (534mg)
During H2O	1013mL/hr (2025mL)	1013mL/hr (2025mL)	608mL/hr (1215mL)
Post	52g carb, 16g pro, 10g fat	75g carb, 23g pro, 15g fat	82g carb, 25g pro, 16g fat

Workout B: 45-Min Easy Run (18°C, 1.5h before, fasted-eligible)

Phase	Elena (52kg)	Marcus (75kg)	Sarah (82kg)
Pre	78g carb, 8g pro, 5g fat	113g carb, 11g pro, 5g fat	123g carb, 12g pro, 5g fat
During	18.0g/hr (14g total)	15.0g/hr (11g total)	10.5g/hr (8g total)
Post	52g carb, 16g pro, 10g fat	75g carb, 23g pro, 15g fat	82g carb, 25g pro, 16g fat

Pre-workout is "snack" type (1.5h before): 0.15 g/kg protein, 5g flat fat. If fasted: skip pre, post carbs ×1.2, post protein 0.35 g/kg.

Workout C: 4-Hour Bike Ride (28°C, 3.5h before)

Phase	Elena (52kg, high gut)	Marcus (75kg, mod gut)	Sarah (82kg, low gut)
Pre	182g carb, 13g pro, 21g fat, 550mg Na, 338mL	263g carb, 19g pro, 30g fat, 550mg Na, 488mL	287g carb, 21g pro, 33g fat, 400mg Na, 533mL
During	108.0g/hr (432g total)	90.0g/hr (360g total)	63.0g/hr (252g total)
During Na	916mg/hr (3663mg)	916mg/hr (3663mg)	327mg/hr (1307mg)
During H2O	1238mL/hr (4950mL)	1238mL/hr (4950mL)	743mL/hr (2970mL)
Post	62g carb, 16g pro, 10g fat	90g carb, 23g pro, 15g fat	98g carb, 25g pro, 16g fat

---

Appendix B: Implementation Reference

Edge Function

Function	Lines	Purpose
`calculatePreWorkoutMacros()`	120-185	Pre-workout (v4 linear formula)
`getDurationCarbBand()`	197-214	TABLE 1 lookup
`getGutTrainingMultiplier()`	222-227	TABLE 3 multipliers
`getSportCarbCeiling()`	232-237	TABLE 2 ceilings
`calculateDuringWorkoutCarbRate()`	248-285	During carbs (4-step algorithm)
`calculateDuringWorkoutHydration()`	290-324	During sodium + hydration
`calculatePostWorkoutCarbs()`	334-346	Post carbs

Function	Lines	Purpose
`calculatePostWorkoutProtein()`	352-358	Post protein
`calculatePostWorkoutFat()`	363-365	Post fat
`calculatePostWorkoutHydration()`	371-399	Post sodium + hydration
`calculateMacrosV3()`	607-756	Single-sport orchestrator
`calculateBrickMacrosV3()`	844-1032	Brick orchestrator

Dart Client

File	Purpose
`macro_generation_service.dart`	Single-sport edge function calls + parsing
`brick_macro_service.dart`	Brick edge function calls + multi-phase parsing
`llm_nutrition_plan_service.dart`	LLM integration with fallback macro estimation

Bug Fixes Applied in v4

1. **Pre-workout protein/fat field mismatch** — Dart parser now reads `pre_run_protein_g` and `pre_run_fat_g` (previously read wrong names, causing silent 0 values)
2. **Brick missing sweat data** — `brick_macro_service.dart` now sends `sweat_rate_category` and `sweat_sodium` from user profile (previously omitted, defaulting to 'medium')
3. **LLM fallback alignment** — `llm_nutrition_plan_service.dart` fallback formulas now match v4 algorithm (previously used hardcoded 2 g/kg pre, 30/45/60 during)

Appendix C: v3-to-v4 Migration Notes

If maintaining backward compatibility with v3-era clients:

- The edge function endpoint name is still `generate-macros-v3` (no rename needed)
- Response field names are unchanged
- The `algorithm_version` field still returns `"v3"` (the algorithm is named v3 in the edge function but this doc calls it v4 to distinguish from the original spec)
- Input fields are additive: old `time_before_run_min` is still accepted for backward compatibility, but `hours_before` is the primary field